

MS-CLS: An Effective Partitioning and Placement Metaheuristic for Wafer-Scale Physics Modeling

Canhui Luo, Zhouxing Su, and Zhipeng Lü

Abstract—Finite element method (FEM) is one of the most successful numerical approaches for solving partial differential equations and has been widely applied in various fields. The basic idea of the FEM is to discretize the target space into smaller and simpler parts, which are solved by algebraic or ordinary differential equations. Simultaneously, a substantial number of decomposed tasks incur expensive computational overhead. The prevailing approach adopted to expedite FEM simulations entails harnessing the power of massively parallel computing and hardware acceleration techniques, which usually involve tackling two challenging combinatorial optimization problems: partitioning and placement. The partitioning models complex physical systems as the assembly of a series of parallelizable tasks. The subsequent placement determines the mapping from computational tasks to parallel processors and optimizes the overall simulation overhead, such as latency and interference, which result from the dependencies or interactions in the data flow. Similar to the successful application of hardware acceleration for boosting the training of deep neural networks in the ISPD 2020 contest, a new challenge for exploiting the power of a wafer-scale engine to accelerate the finite element method was proposed in ISPD 2021. First, we need to partition the target space into non-uniform grids according to the heatmap to maximize the accuracy. Then, the computing task corresponding to each grid needs to be placed onto a 2D array of 848,000 processors to minimize the total communication overhead. In this paper, we propose an effective metaheuristic algorithm named MS-CLS for tackling this challenging problem by introducing a Merging-Splitting-based heuristic for partitioning and a Cluster-based Local Search metaheuristic for placement. Compared with the records of the ISPD 2021 contest, MS-CLS outperforms all the qualified teams in the contest on all the benchmark datasets in a short computational time. Moreover, similar techniques proposed in this work can be used on other hardware parallel acceleration applications.

Index Terms—Scheduling and task partitioning, Placement and routing, Heuristic search, Wafer-scale physics modeling

I. INTRODUCTION

ANALYTICAL solutions of complex nonlinear partial differential equations (PDEs) in engineering computations are usually very difficult to find, but approximate numerical solutions are sufficient in most cases. Finite Element Method (FEM) is such a mature numerical method for engineering modeling and solving PDEs [1]. The application of FEM is inextricably associated with the development of computer technology. It is initially used in plane stress analysis [2]. The growing computing power makes it capable of solving extremely complex problems, and it has been widely applied

to many fields, such as biomedicine imaging [3], mechanical analysis [4], manufacturing technology [5], etc.

The core concept of FEM is to subdivide a complex target space into numerous small and simple pieces (the finite elements), usually triangles for 2D applications and tetrahedra for 3D geometries. FEM describes and solves the field solution for each finite element with comparatively simple linear or quadratic equations. Then, it obtains the approximate solution for the complete system by assembling the results of finite elements and their adjacency [6].

The accuracy of solutions obtained by FEM improves as partitioning fineness increases, but the computation complexity grows remarkably as a consequence, since there will be more elements and even more communication among elements. In order to improve the overall modeling accuracy and reduce the computational cost as much as possible, FEM usually partitions the target space into non-uniform grids. In most practical scenarios, the precision requirement of different regions in the target space varies. For instance, we may pay more attention to the regions with higher posteriori local error estimation. Apparently, we should spend more computing power for the regions with higher precision requirement, and the fine-grained partitioning will be only adopted in such regions. Although a few additional calculations are required to resolve the resolution differences, the amount of calculation is substantially reduced comparing to applying the finest partitioning to the entire space, while there is no significant loss in accuracy.

In addition to discretizing the target space, improving the computational efficiency of the tasks corresponding to finite elements is another important research topic. Since the FEM computing for finite elements is highly parallelizable, there has been some attempts in recent years which integrate parallel computing techniques such as GPU acceleration into the FEM implementations [7]–[9]. However, the performance enhancement is largely restricted due to the limited power of the general-purpose devices like GPU. Recently, ISPD 2021 initiated a challenge regarding partitioning and placement to exploit the power of a wafer-scale engine (WSE) on the FEM acceleration [10]. The challenge is closely related to multiple classic NP-hard problems and practical applications such as partitioning [11], placement [12], covering [13], and assignment [14]. Briefly speaking, it aims to partition the target space into small regions, and place each computing task corresponding to a region onto WSE.

The contestants in the ISPD 2021 contest (the k -th place in the contest is denoted as ISPD# k) introduced various effective algorithms [15] to solve this challenging problem, and a state-of-the-art algorithm was proposed in [16]. All the

The authors are with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China, 430074. E-mail: {luocanhui, suzhouxing, zhipeng.lv}@hust.edu.cn.
(Corresponding author: Zhouxing Su.)

algorithms adopt a sequential optimization from partitioning to placement, except for ISPD#5, which iterates between two parts and stops when the difference between the accuracy score and the connectivity score is less than a predetermined threshold. In partitioning, all algorithms are based on merging and splitting operations and can be classified as bottom-up (by most algorithms) or top-down (by ISPD#6) frameworks depending on the operation sequence. The bottom-up method first partitions with a uniformly fine-grained level on the target space and then achieves a non-uniform partitioning result through merging operations and reduces computational resources. In contrast, the top-down procedure starts with coarse-grained partitioning and refines the partitioning layer-by-layer to improve accuracy. The placement optimization is further divided into initial placement and refinement. There are both heuristic and analytical algorithms for initial placement. [16] employs a connection-based heuristic that takes into account the corresponding spatial locations of the tasks and their adjacency. ISPD#6 iteratively optimizes the placement by conjugate gradient descent based on the mathematical programming model. ISPD#1 and ISPD#4 model the task placement as a cell placement optimization problem in Very Large Scale Integrated (VLSI) circuits and adopt ePlace [17] and NTUplace3 [18], respectively, which are classic analytical solvers for global placement in VLSI. Then, based on the greedy swap heuristic, i.e., moving a task from one processor to another or swapping the processors corresponding to two tasks, the initial placement is further refined for improvement.

In this paper, we focus on the problem introduced in the ISPD 2021 contest which is essential to the physics modeling powered by WSE.

Our main contributions are as follows:

- We formulate the mathematical model of the partitioning and placement problem for FEM acceleration, and analyze its relationship with several classical NP-hard combinatorial optimization problems.
- We propose an effective Merging-Splitting and Cluster-based Local Search (MS-CLS) algorithm to tackle the partitioning and placement problem for FEM acceleration.
- We propose a non-uniform merging-splitting-based partitioning algorithm which divides the target space into a number of small regions according to a given heatmap and computing resource limit.
- We propose a clustering-based local search metaheuristic for placement optimization. It consists of a novel constructive heuristic based on spatial recursive clustering, and a local search with swap and move neighborhoods is employed to refine the placement.
- We conduct experiments to compare the proposed algorithm with the winners of the ISPD 2021 contest and other contest solvers. MS-CLS can improve the best-known results of all the competitors on all the datasets used in the contest within a short run time.

The remainder of the paper is organized as follows. In Section II, we clarify the process of physical modeling for FEM using WSE. We introduce the detailed problem for-

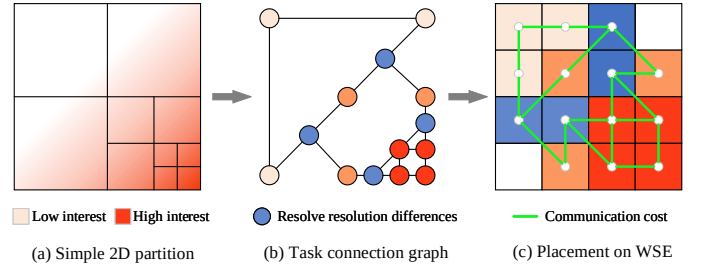


Fig. 1. Example of FEM in 2D space from partitioning to placement. (a) The target space with different target resolution distributions and corresponding partitioning results. The dark regions represent high target resolutions, and the fine-grained partitioning is used. (b) The task connection graph corresponds to the partitioning result. Each circle represents a computation task. Each blue circle acts as an adapter to resolve resolution differences, while each of other circles corresponds to a specific region of the target space. (c) The mapping results of the computation task graph to the physical computing engine. The square array means an engine with $4 \times 4 = 16$ processing elements (PEs). The colored PEs are responsible for computing tasks corresponding to the same color in the task graph (b). The blank PEs do not perform task computations but can be relays for forwarding data packets. The green links represent the communication cost between PEs during computation, and shorter wire length indicate lower cost.

mulation and analysis in Section III. Then, we describe our proposed MS-CLS algorithm in detail in Section IV, and give the experimental results and discussion in Section V, followed by conclusions.

II. WAFER-SCALE PHYSICS MODELING

Hardware acceleration for massive parallel computing has been an important topic in the fields of distributed computing and electronic design automation in recent years. What makes things more interesting is that hardware acceleration can be used to design and test the hardware themselves. For example, Google developed a deep reinforcement learning approach for chip floorplanning of a recent-generation tensor processing unit accelerator by its older version [19]. IBM designed partitioning and placement algorithms for hardware-accelerated functional verification [20], [21]. The underlying problems of the hardware acceleration consist of how to partition the computing task into smaller ones and how to assign them to a large number of processors, so that the tasks can be done faster and the communication overhead is low.

The recently developed Cerebras CS-1 is a novel supercomputer with massive homogeneous multiprocessors powered by a wafer-scale engine (WSE) [10]. It consists of 848,000 identical programmable execution cores called processing element (PE), each of which has 48kB SRAM, and a router for interconnection with other cores. CS-1 is very suitable for stencil problems since all cores are arranged in a standard grid. In fact, WSE is 10,000 times faster than a GPU node and 200 times faster than any supercomputer cluster [22]. In addition, WSE has also been successfully applied to accelerate the training procedure of the deep neural network [23]–[25].

The wafer-scale physics modeling consists of two stages: Partitioning the target space and placing the corresponding computing tasks to PEs on WSE. In detail, the target space is first partitioned into multiple square (cubic for 3D space) regions of different sizes according to the target resolution.

Fig. 1(a) illustrates a simple example of a non-uniform partitioning of a 2D space. It can be regarded as tiling the target space with squares, and the area covered by each square is processed by unit computing power. The larger the area of a square is, the lower its resolution would be. Then, as shown in Figs. 1(b) and 1(c), a series of computing tasks corresponding to squares with mutual communications, including solving numerical equations on a single element and processing resolution differences, are assigned to the array of processors. In order to achieve high accuracy and performance with the limited number of PEs and communication bandwidth, it is important to make appropriate partitioning of the space to fit the target resolutions, as well as proper placement of the partitions to the array of PEs to reduce the communication cost.

Depending on the specific application scenario, the parallel task graph computing system usually involves the optimization of performance metrics such as makespan [26], throughput [27], and energy efficiency [28]. In this work, we do not need to consider data dependency of tasks on the processor, as the FEM simulation is performed in a synchronized manner and the data are only transmitted between consecutive cycles. Thus, our objective is to minimize the communication cost among directly connected tasks. Task placement directly determines the communication cost, and placing adjacent tasks to distant processors generally leads to high latency and congestion. For simplicity, the communication cost in placement optimization is measured by the total half-perimeter wire length among the assigned PEs for all tasks instead of the actual routing.

III. PROBLEM DESCRIPTION AND ANALYSIS

We introduce some basic concepts related to partitioning and placement in this section. Then, the problem formulation including the evaluation function and constraints is presented¹, followed by the problem analysis. For convenience, we define the problem of **FEM** with **Partition** and **Placement** optimization as FEMPP hereinafter and focus on the 3D case unless otherwise specified.

A. Preliminaries

In FEMPP, the *target space* S is defined by a cuboid bounding region, i.e., $S = \{(x, y, z) : 0 \leq x \leq X, 0 \leq y \leq Y, 0 \leq z \leq Z\}$ in Cartesian coordinate system. The *heatmap* $H : (x, y, z) \rightarrow [0, 1]$ is a function that maps point (x, y, z) in the target space to a *target resolution*, which is the desired computing power per unit volume. In practice, the target space is discretized by a regular grid of sample points with a unit step U , so that the heatmap can be defined by a 3D array R where each entry R_{ijk} gives the target resolution of grid point (iU, jU, kU) , and the target resolution elsewhere is evaluated via trilinear interpolation.

On WSE, all PEs are arranged in a 1060×800 grid. Each PE can be programmed to be a mesh PE, an adapter PE, or an

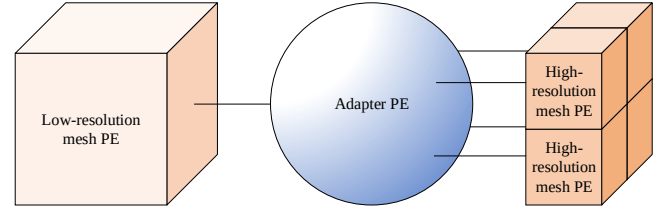


Fig. 2. The communication structure between PEs in 3D scenario. The left mesh PE with low-resolution communicates with four high-resolution mesh PEs on the right through an adapter PE. The adapter PE is used to resolve resolution differences between mesh PEs.

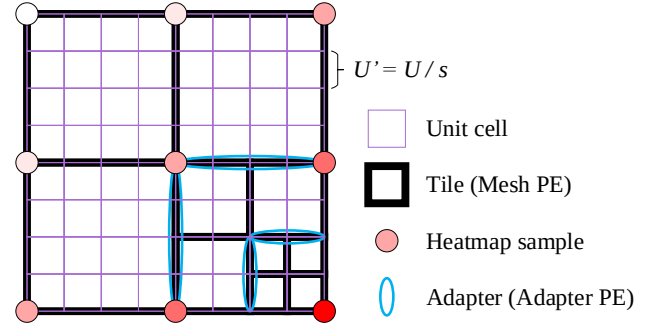


Fig. 3. An example of partitioning a 2D target space. The initial size of the heatmap is 2×2 , and it is refined with an inverse sampling step of $s = 4$ to form 8×8 fine-grained sampling unit cells to support flexible partitioning. The smallest tile in the bottom right corner has a high actual resolution, while the tile in the upper left corner has a low actual resolution. Adapters (marked as blue ellipses) must be assigned among tiles with different resolutions in the placement stage.

unused PE. In detail, a *mesh PE* solves differential equations in $M \times M \times M$ mesh points where $M = 10$ in the current hardware specification. A mesh PE must cover a cubic region in the target space due to the hardware restriction. It is easy to see that the *actual resolution* increases as the size of the cube decreases. An *adapter PE* resolves mesh resolution differences at the boundary between regions with different resolutions by interpolation. As illustrated in Fig. 2, a low-resolution mesh PE and four (two) high-resolution ones must be linked by an adapter PE in 3D (2D) case. This adjacency requirement implies that the actual resolution on the high-resolution side is exactly twice as much as the low-resolution one. Finally, an *unused PE* does not perform any calculation but only forwards packets.

The partitioning of the target space can be represented by a seamless tiling as illustrated in Fig. 3, where each *tile* should be assigned to a single mesh PE in the placement stage. In order to make it easy to align the tiles while supporting flexible partitioning, the heatmap unit step U can be further divided into s ones, where *inverse sampling step* s is a positive integer to be determined by the algorithm. As a result, the target space is discretized by a regular grid of cells, whose edge length is $U' = \frac{U}{s}$. The edge length of each tile must be $2^n MU'$ where n can be any non-negative integer. Meanwhile, the corner of each tile must be at (iU', jU', kU') , where i, j, k are non-negative integers. The adjacent tiles directly communicate with each

¹The final problem model adopted in the ISPD 2021 contest is slightly different from [10]. Our description is consistent with the contest.

other if their edge length is the same. Otherwise, an *adapter* is needed and must be assigned to an adapter PE in the placement stage.

B. Objective Function

The objective of FEMPP is to find a valid mapping that maximizes fidelity in the regions of interest while maximizing the emulation rate. We denote S^d as the discretized target space under the inverse sampling step s . Let $T = (t_0, t_1, t_2, \dots, t_n)$ denote the partitioning result, i.e., the set of tiles which fill the discretized target space S^d , and let $P = (e_0, e_1, e_2, \dots, e_n)$ denote the placement result, where e_i signifies the PE to which the task of tile t_i is mapped. The objective function $f(s, T, P)$ is a trade-off between the accuracy of the partitioning $f_a(s, T)$ and the connectivity of the placement $f_c(T, P)$. Both parts are normalized and the objective is to maximize the smaller value between $f_a(s, T)$ and $f_c(T, P)$ as presented in Eq. (1).

$$\max_{(s, T, P)} f(s, T, P) = \min\{f_a(s, T), f_c(T, P)\} \quad (1)$$

The detailed definitions of the accuracy objective $f_a(s, T)$ and connectivity objective $f_c(T, P)$ are as follows:

1) *Accuracy Evaluation*: The objective of the partitioning is to make the actual resolution meet the target resolution as much as possible, in terms of both overall and worst cases. Let $S_t^d \subset S^d$ represent the space covered by tile $t \in T$ whose edge length is l_t and volume is $V_t = l_t^3$. Let $A(p)$ denote the actual resolution of point $p \in S^d$. Also recall that $H(p)$ is the target resolution of point $p \in S^d$. Without loss of generality, we assume that for each tile t , the actual resolution is the same for all $p \in S_t^d$, which is denoted as $A_t = \frac{MU'}{l_t}$. Then, the accuracy score of tile t can be calculated by Eq. (2)².

$$f_a(s, t) = \frac{\oint_{S_t^d} H(p) dp}{\oint_{S_t^d} A(p) dp} = \frac{\oint_{S_t^d} H(p) dp}{A_t V_t} \quad (2)$$

To encourage evenly distributing the computing power, the maximum ratio of the target resolution to the actual one is used as the discount factor d , which is calculated by Eq. (3).

$$d = \max\{1.0, \max_{p \in S^d} \left\{ \frac{H(p)}{A(p)} \right\}\} \quad (3)$$

In addition, the accuracy is normalized by the number of available PEs which is denoted by N . Thus, the final accuracy score can be expressed as Eq. (4).

$$f_a(s, T) = \frac{1}{dN} \sum_{t \in T} \frac{\oint_{S_t^d} H(p) dp}{A_t V_t} \quad (4)$$

2) *Connectivity Evaluation*: For each tile or adapter, we need to assign it to a PE to execute its computational task. We denote the set of these two kinds of tasks by T^* and the set of available PEs by E . Let $e_t \in E$ be the assigned PE for task $t \in T^*$. The set of tasks which needs to communicate with task t is denoted as $C_t \subset T^*$. For each

adjacent tile or adapter $t' \in C_t$, $W_{e_t e_{t'}}$ represents the half-perimeter wire length (HPWL) of the connection between PEs e_t and $e_{t'}$. The communication overhead of the placement is measured by the $L_{1.5}$ -norm of HPWL, which disproportionately penalizes long connections comparing to the L_1 -norm but not as severely as the L_2 -norm [10]. Besides, by analyzing the network utilization of WSE, a fixed parameter $(100N)^{\frac{2}{3}}$ is used to normalize the connectivity score. Therefore, the final connectivity score is calculated as Eq. (5).

$$f_c(T, P) = \left(\frac{100N}{\sum_{t \in T^*} \sum_{t' \in C_t} W_{e_t e_{t'}}^{\frac{2}{3}}} \right)^{\frac{2}{3}} \quad (5)$$

C. Constraints

A valid solution must satisfy the following constraints regarding the partitioning and placement, respectively.

1) Partitioning Constraints:

- **Seamless Tiling**. The target space must be completely filled by tiles without overlapping. Apparently, it implies that the inverse sampling step s must respect that $\frac{X}{U}s \equiv \frac{Y}{U}s \equiv \frac{Z}{U}s \equiv 0 \pmod{M}$.
- **Subtle Gradient**. The ratio of the edge lengths of two adjacent tiles must be 1:1 or 1:2.
- **Tile Alignment**. Each edge of each tile is in parallel with an axis and each pair of adjacent tiles share at least one corner point.
- **Adapter Requirement**. If a tile is aligned to four tiles with half of its edge length, there must be an adapter connecting to each of the five tiles as presented in Fig. 2.
- **PE Number**. The total number of tiles and adapters must not exceed the total number of PEs.

2) Placement Constraints:

- **Single Assignment**. Each tile or adapter must be assigned to exactly one PE, i.e., placed on WSE.
- **Single Usage**. Each PE can only execute no more than one task corresponding to a tile or an adapter.

D. Problem Analysis

For the partitioning stage, it is very important to determine an appropriate inverse sampling step s , since it is the foundation of all partitioning decisions. A small s usually results in that only a small amount of PEs are used to cover the entire target space. However, as the accuracy is eventually normalized by the total number of PEs in Eq. (4), a low utilization of computing power usually leads to poor accuracy score. On the contrary, a large s may require more PEs than the hardware limit, i.e., it will violate the PE number constraint. Although we can obtain a feasible partitioning by reducing the resolution of PEs to cover a larger region, it comes with a larger discount factor d and introduces huge penalties. It is noteworthy that the accuracy score only takes the tiles into account but not the adapters, so the resolution difference between adjacent tiles should be avoided as much as possible to reduce the use of adapters. In summary, it encourages to use as many mesh PEs as possible in the partitioning stage to achieve higher accuracy.

²Since the heatmap is given by interpolation, the integral of the target resolution over the covering cuboid can be simplified as the product of the average target resolution of the corner points with the volume of the covering cuboid.

The solution of the partitioning stage has a huge impact on the complexity and upper bound of the placement optimization. Apart from the total number of tiles and adapters, the proportion of adapters is also an important factor to the connectivity score. Except for the ones on the boundary of the target space, each tile connects to six tiles or adapters (one per face), while an adapter only connects to five tiles (four high-resolution tiles and one low-resolution tile). Thus, the greater the proportion of adapters is, the sparser the connection topology will be. Considering that each PE can only communicate with four surrounding PEs (less than the degree of a tile or adapter) on WSE without additional penalties, the communication cost of a topology is usually low, which results in a high connectivity score.

Regarding the placement stage, the critical issue is to map a topological graph (task network) into another one (PE network). An interesting fact is that the relative spacial positions are hidden in both topological graphs. In other words, the tasks which are close to each other in the geometric space are only a few hops away in the topological graph, and so do the PEs. Therefore, the geometric distribution of the vertices can be used as a valuable hint to the placement. Thus, the problem can be regarded as performing dimensionality reduction from a 3D structure to a 2D one.

IV. MS-CLS ALGORITHM

A. General Framework

We propose a merging-splitting and clustering-based local search algorithm named MS-CLS to tackle FEMPP. On the one hand, it employs merging and splitting operations to iteratively improve the solution quality of partitioning. On the other hand, it integrates a clustering-based heuristic to generate the initial placement and adopts a local search algorithm based on swap and move operations to iteratively improve the placement.

Fig. 4 presents the overall framework of MS-CLS. It tackles the partitioning and placement sub-problems sequentially. The partitioning stage starts from the inverse sampling step estimation. Then, it tries to find a feasible solution under a series of potential inverse sampling steps. Starting from the estimated inverse sampling step, the proposed algorithm partitions the entire target space with the tiles whose resolution is 1.0. We first obtain an illegal solution due to the fact that the number of required PEs are usually far beyond the available ones. However, there are many regions whose target resolution is lower than 1.0, so we reduce their actual resolution by merging adjacent tiles to reduce the number of used PEs. If the number of required PEs still exceeds the limit after the merging operations, MS-CLS reduces the inverse sampling step and repeats the above procedures. Otherwise, we use the surplus PEs to further optimize the accuracy through the splitting operations.

The output of the partitioning stage which describes the connections between tiles and adapters is the input of placement optimization. MS-CLS generates the initial placement by performing recursive clustering on tiles in the 3D target space and mapping them to the PE array. Then, it employs a local search algorithm which integrates swap and move operations

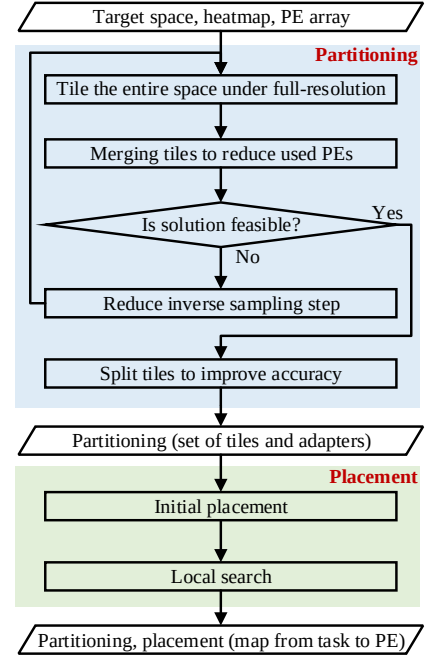


Fig. 4. Framework of the proposed MS-CLS algorithm.

to iteratively improve the quality of the placement until the time limit is reached.

B. Partitioning 1: Inverse Sampling Step Estimation

In the partitioning stage, it is usually easy to obtain feasible solutions under small inverse sampling step s , but larger s enables more flexible partitioning. A straightforward idea is to gradually increase s until we fail to find a better solution, but it is very time-consuming to try all possible values. Furthermore, binary search is inapplicable because the accuracy is penalized by discount factor d and becomes a non-monotonic function of the inverse sampling step s . Based on these reasons, MS-CLS first estimates the required number of PEs of a sufficiently large s under a fixed d , and then it reduces s until the available PEs on WSE are not exceeded. In MS-CLS, we assume that the discount factor $d = 2.0$ when performing the inverse sampling step estimation. It is reasonable since the solution quality will be poor if the gap between the actual and target resolutions is large, i.e., $d > 2.0$. However, it is almost impossible to find a solution which satisfies the target resolution everywhere in the target space, i.e., $d = 1.0$.

Let G_i be the total number of $U \times U \times U$ grids in the target space whose target resolution is between interval $(2^{-i}, 2^{1-i}]$ for any positive integer i . In order to guarantee $d \leq 2.0$, each G_i grid should be covered by tiles whose resolution is at least 2^{-i} . Thus, the estimated minimum number of required mesh PEs is $N' = \sum_{i=1}^{+\infty} G_i 2^{-3i}$. The growth of the demand for PEs is approximately proportional to the third power of the growth of the number of grids in each dimension. Recall that N represents the number of available PEs, and we can estimate that the number of grids in each dimension can be increased to $(\frac{N}{N'})^{\frac{1}{3}}$ times of the original one. So, we estimate the inverse sampling step by $s' = M(\frac{N}{N'})^{\frac{1}{3}}$.

Algorithm 1 Merging-splitting-based Partitioning

Input: Target space S , heatmap H , PE number N
Output: A set of tiles T

```

1:  $s \leftarrow$  estimated inverse sampling step
2: repeat
3:    $T \leftarrow$  1.0-resolution tiles that fill the entire  $S$  under  $s$ 
4:   Perform merging operations on  $T$ 
5:   /*  $N_r(t)$  is the required PE number to split tile  $t$  */
6:    $T_d \leftarrow \{t \in T : \max_{p \in t} \frac{H(p)}{A_t} = d\}$ 
7:   while  $d > 1.0$  and  $N - |T| \geq \sum_{t \in T_d} N_r(t)$  do
8:     Split all tiles  $t \in T_d$ 
9:     Update  $d$  and select new  $T_d$  as line 6
10:  end while
11:   $t^* \leftarrow \arg \max_{t \in T} \{(\sum_{i=0}^7 f_a(s, t_i) - f_a(s, t))/N_r(t)\}$ 
12:  while  $N - |T| \geq N_r(t^*)$  do
13:    Split tile  $t^*$ 
14:    Select new  $t^*$  as line 11
15:  end while
16:   $s \leftarrow$  next smaller  $s$ 
17: until  $T$  is feasible or the time limit is reached

```

If $s > s'$, it is very hard to find any feasible solution which satisfies $d \leq 2.0$. However, simply adopting $s = \lfloor s' \rfloor$ may make it impossible to satisfy the seamless tiling constraint or the required number of PEs exceeds the available ones as we do not consider the adapters. Thus, we have to iterate through every potential inverse sampling step $s \in \{i \in [M, s'] : \frac{X}{U}i \equiv \frac{Y}{U}i \equiv \frac{Z}{U}i \equiv 0 \pmod{M}\}$ in descending order.

C. Partitioning 2: Merging and Splitting Operations

For each specific inverse sampling step, the target space is discretized into a regular grid of unit cells so that the proposed partitioning algorithm can be performed. MS-CLS iteratively performs bottom-up (merging) and top-down (splitting) operations to optimize the current solution, as described in Algorithm 1.

First, we relax the PE number constraint and tile the entire target space with 1.0-resolution tiles (line 3). However, there are a lot of regions where the current actual resolution exceeds the target resolution, which wastes computing resources. Thus, we try to reduce the number of tiles by merging adjacent tiles to fix the PE number constraint violation in the initial partitioning while improving the average accuracy score per tile $f_a(t)$ (line 4). In detail, the merging operation detects the regions covered by $2 \times 2 \times 2$ tiles with the same resolution and replaces them with a double-edge-length tile. Obviously, a successful merging operation reduces seven tiles. Even if we take adapters into account, a merging operation still reduces the number of required PEs, since each face of a tile only introduces at most one adapter to resolve the resolution difference after merging.

To avoid other constraint violations, we only perform a merging operation when the subtle gradient and tile alignment constraints are satisfied. In other words, the resolution of the tile before merging must not be lower than its neighbors', and each face of the merged tile is aligned with one equal-resolution tile or four double-resolution ones. Fig. 5 illustrates

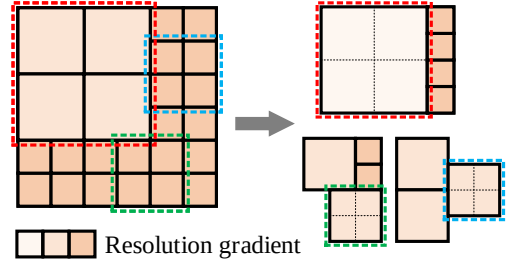


Fig. 5. Illustration of invalid merging operations in a 2D case. Merging situations similar to the red box result in *Subtle Gradient* constraint violations, and those in the green and blue boxes result in *Tile Alignment* constraint violations, as described in Section III-C.

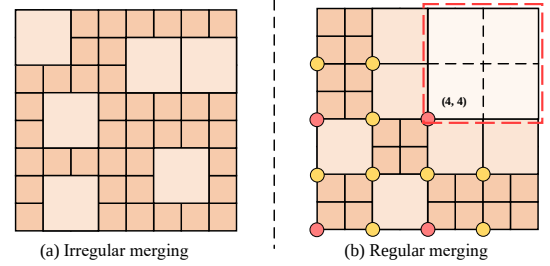


Fig. 6. Comparison of irregular and regular merging. (a) The partitioning cannot be further merged anywhere. (b) The red and yellow dots indicate the coordinates of the tiles that may be merged, and the red dashed box indicates that we can merge the tiles into a single tile whose edge length is doubled and the resolution is halved.

some constraint violations in a 2D scenario. Merging the tiles in the red box will result in a 1:4 resolution ratio between adjacent tiles, which violates the subtle gradient constraint. Meanwhile, merging the tiles in either the green or blue boxes violates the tile alignment constraint.

In addition, irregular merging will mess up the distribution of tiles, making it difficult to merge further as shown in Fig. 6. Therefore, if the current merging operation is to merge the tiles with edge length u into a tile whose edge length is $2u$, it will be ignored unless the left-bottom tile is located at $(2ui, 2uj)$ where i and j are non-negative integers.

Regarding the influences on the accuracy score, the merging operation halves the actual resolution of the corresponding region which incurs high penalties when it is lower than the target resolution. Specifically, such penalties mainly result from the discount factor d . Hence, we follow the assumption in inverse sampling step estimation and only perform the merging operations which satisfy $d \leq 2.0$.

MS-CLS iteratively merges tiles in descending order of actual resolution, i.e., it tries to merge the tiles whose resolution is 1.0, then 0.5, and so on. At each iteration, we traverse the tiles of the corresponding resolution and merge the ones which meet all the aforementioned requirements. The trials stop until no more admissible merging operations can be performed. If it fails to produce a feasible solution, i.e., the total number of tiles and adapters still exceeds the number of available PEs, the proposed algorithm reduces the inverse sampling step and repeats the iterative merging (lines 3-4).

Once a feasible solution is obtained after the merging

phase, there are usually unused PEs. According to the problem analysis in Section III-D, they should be used as much as possible. So we adopt splitting operations to optimize the current solution. As the reverse operation of merging, a splitting operation replaces a single tile with eight half-edge-length ones. Similar to merging, the subtle gradient and tile alignment constraints must be maintained in the splitting phase.

The splitting phase is further divided into two subroutines, where the first one focuses on reducing the discount factor d (lines 6-10 in Algorithm 1) and the second one maximizes the resolution improvement per used PE (lines 11-15 in Algorithm 1). Since d penalizes the overall accuracy score, reducing it can dramatically improve the solution quality. Therefore, we first find all the tiles whose target-to-actual resolution ratios are equal to d , and check whether there are enough vacant PEs to split them. If so, we perform the splitting operations on them to reduce d . Then, we move to the new bottleneck of insufficient accuracy. Otherwise, we maximize the accuracy score improvement as the primary objective and minimize the number of adapters as the secondary objective. In detail, assuming that tile t is split into $2 \times 2 \times 2$ tiles $t_0, t_1, \dots, t_7$, the improvement to the accuracy score is $\sum_{i=0}^7 f_a(t_i) - f_a(t)$. Meanwhile, the splitting operation introduces seven mesh PEs and $[-6, 6]$ adapter PEs since we may add or remove an adapter on each of the six faces of the cube. As a result, our evaluation metric encourages utilizing mesh PEs and reducing adapter PEs. Then, the proposed algorithm iteratively performs the best splitting operation according to this metric until there is no enough vacant PEs, and finally returns the resulting solution for the partitioning stage.

D. Placement 1: Cluster-based Initial Placement

Once the partition is fixed, the wire length optimization in the placement stage can be modeled by the quadratic assignment problem (QAP) [29]. Formally, let p_{te} be a Boolean variable where $p_{te} = 1$ indicates task t is assigned to PE e , we can formulate the placement problem as model (QAP).

$$(QAP) \min \sum_{t \in T^*} \sum_{t' \in C_t} \sum_{e \in E} \sum_{e' \in E} W_{ee'} p_{te} p_{t'e'} \quad (6)$$

$$s.t. \sum_{e \in E} p_{te} = 1, \forall t \in T^* \quad (7)$$

$$\sum_{t \in T^*} p_{te} = 1, \forall e \in E \quad (8)$$

$$p_{te} \in \{0, 1\}, \forall t \in T^*, \forall e \in E \quad (9)$$

In model (QAP), objective function (6) minimizes the total wire length by adding the wire length of PEs e and e' if a pair of adjacent tasks t and t' are placed on them correspondingly. Constraints (7) ensure that each task is assigned exactly to one PE. Constraints (8) restrict that each PE can be occupied by exactly one task (one may need to add some dummy tasks to make the numbers of tasks and PEs matched).

The state-of-the-art algorithms may take hours for solving the QAP instances with thousands of tasks [29]. However, a typical FEMPP instance contains millions of tasks, and the computational time is huge and unacceptable in real-world

applications. In contrast to finding the optimal solution for QAP, it is relatively easy to generate a feasible placement. For example, just randomly assigning the tasks, i.e., tiles and adapters, to any idle PE is able to obtain a feasible solution. An improved approach is to greedily place the tasks one by one according to the wire length from their placed neighbors. However, both methods pay too much attention to the details and neglect the whole picture of the tasks. Inspired by the typical design flow of very-large-scale integrated circuits (VLSI), where the global placement determines the coordinates of the cells and the detailed placement resolves overlaps and misalignment [30]–[33], we propose an initial placement algorithm whose main idea is to make use of the spatial distribution of the tasks. The main process is depicted in Fig. 7. It further reformulates QAP to the generalized assignment problem (GAP) [34] and performs recursive clustering in a divide-and-conquer paradigm.

There are mainly two challenges which make the resulting QAP in FEMPP hard to solve. On the one hand, it seeks for the perfect match from 10^6 tasks to 10^6 PEs, which means that the dimension of the solution vector can be 10^{12} . On the other hand, the quadratic terms (placing task t to PE e while placing task t' to PE e') greatly deteriorate the convexity of the optimization model. In order to overcome these difficulties, we relax the perfect matching constraint, i.e., we assign each task to a cluster of PEs instead of a single PE. Moreover, we change the objective from “task t prefers to be placed close to the PE for task t' ” (quadratic term) to “task t prefers to be in a certain cluster” (linear term) by utilizing the geometric information.

1) *Clustering Pattern*: How do we define the preference of a task to a cluster? One natural way is to first determine the core of each cluster and then define the distance between the task and the core as the preference. Now the problem becomes where to locate the cores of clusters. In fact, a tile corresponds to a region in the target space, so we represent its position by the coordinates of its center. An adapter is in the middle of a low-resolution tile and four high-resolution tiles, and its position is set to the centroid of these connected tiles. We use parallel projection to map all tasks onto a plane. Different centers of projection will be evaluated and the one with the largest projected area is adopted. Specifically, MS-CLS only tries orthographic projection, i.e., the candidate centers of projection are $(0, 0, +\infty)$, $(0, +\infty, 0)$, and $(+\infty, 0, 0)$ and the tasks are projected onto $z = 0$, $y = 0$, and $x = 0$ planes, respectively. Then, we employ the classical Andrew’s algorithm [35] to compute the convex hull of the shadow, and calculate the area of the covering polygon. MS-CLS prefers the direction of the largest projection area, as the overlap of the tasks is the least. To unify the processing, if the projection plane is not $z = 0$, we will exchange the dimensions so that the coordinates of the projected points are (x, y) , as illustrated in Fig. 7(b). Assuming that the coordinates of the tasks are bounded by $\{(X^-, Y^-, Z^-), (X^+, Y^+, Z^+)\}$, in the case of four clusters, we set the cores of the clusters to (X^-, Y^-, Z^-) , (X^-, Y^+, Z^-) , (X^+, Y^-, Z^+) , and (X^+, Y^+, Z^+) , as shown in Fig. 7(c). More discussion about core locations will be presented in Section VI-E.

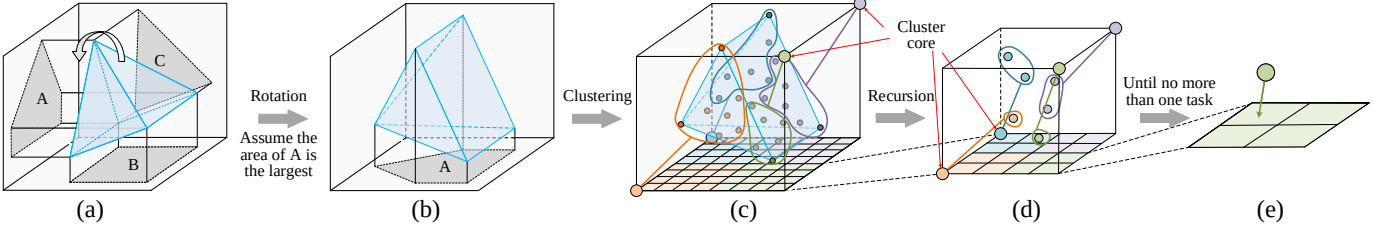


Fig. 7. Clustering pattern and recursive clustering. (a) Find the projection direction with the maximum projected area of the object. (b) Rotate the object so that the direction of the maximum projected area faces downward. (c) Set suitable cluster cores to divide tasks into multiple clusters. (d) Perform recursive clustering for each cluster. (e) The recursion terminates when there is no more than one task in the cluster.

2) *Recursive Clustering*: Given the coordinates of the tasks in the 3D space, MS-CLS recursively divides the tasks into $J \times J$ clusters until there is no more than a single task in each cluster, as shown in Figs. 7(c), 7(d), and 7(e). Each cluster corresponds to a non-overlapping rectangular area of PEs on WSE. In other words, model (*QAP*) is reformulated to a series of GAP, which studies assigning tasks to capacitated containers. Let the cost (or preference) of assigning task t to cluster j be K_{tj} , which is equal to the distance between the core of cluster j and the spatial position of task t . Let Boolean variable q_{tj} denote whether task t is assigned to cluster j . Model (*GAP*) presents the GAP at the root level of the recursive clustering.

$$(GAP) \min \sum_{t \in T^*} \sum_{j=1}^{J^2} K_{tj} q_{tj} \quad (10)$$

$$s.t. \sum_{j=1}^{J^2} q_{tj} = 1, \forall t \in T^* \quad (11)$$

$$\sum_{t \in T^*} q_{tj} \leq N/J^2, \forall j \in \{1, 2, \dots, J^2\} \quad (12)$$

$$q_{tj} \in \{0, 1\}, \forall t \in T^*, \forall j \in \{1, 2, \dots, J^2\} \quad (13)$$

In model (*GAP*), objective function (10) minimizes the total costs of assigning tasks to clusters. Constraints (11) ensure that each task is assigned to exactly one cluster. Constraints (12) restrict that each cluster can contain no more than N/J^2 tasks, where N is the number of PEs and they are evenly distributed to each cluster. Once we solve model (*GAP*), it derives $J \times J$ sub-problems (*GAP_j*), each of which corresponds to a cluster $j = 1, 2, \dots, J^2$. The number of PEs is replaced by $N \leftarrow N/J^2$ and the set of tasks becomes $T^* \leftarrow \{t \in T^* : q_{tj} = 1\}$. In fact, if the number of clusters is the same as the number of PEs, constraints (12) are equivalent to constraints (8).

Although the complexity of the placement optimization has been greatly reduced after the reformulation, solving each sub-problem to optimality is still time-consuming, especially when J is large. Therefore, the proposed MS-CLS implementation performs 4-clustering, i.e., $J = 2$, and solves model (*GAP*) by a round-robin greedy selection algorithm as described in Algorithm 2. Given an array of $\omega \times \eta$ PEs, it is divided into four rectangles which are bounded by $\{(0, 0), (\frac{\omega}{2}, \frac{\eta}{2})\}$, $\{(0, \frac{\eta}{2}), (\frac{\omega}{2}, \eta)\}$, $\{(\frac{\omega}{2}, 0), (\omega, \frac{\eta}{2})\}$, and $\{(\frac{\omega}{2}, \frac{\eta}{2}), (\omega, \eta)\}$, respectively. Obviously, the capacity of each quadrant, i.e., the maximum number of tasks assigned to each cluster, is

Algorithm 2 Recursive Clustering

Input: Task set T^* , coordinates of tasks O , left bottom corner of PE array (χ, φ) , width of PE array ω , height of PE array η

Output: PE for each task e

```

1: if  $|T^*| \leq 1$  then
2:    $e_t \leftarrow$  PE at  $(\chi, \varphi), \forall t \in T^*$ 
3:   return
4: end if
5:  $\{T_0^*, T_1^*, T_2^*, T_3^*\} \leftarrow \text{RoundRobinAssignment}(T^*, O)$ 
6: RecursiveClustering( $T_0^*, O, \chi, \varphi, \lfloor \frac{\omega}{2} \rfloor, \lfloor \frac{\eta}{2} \rfloor$ )
7: RecursiveClustering( $T_1^*, O, \chi, \varphi + \lfloor \frac{\eta}{2} \rfloor, \lfloor \frac{\omega}{2} \rfloor, \eta - \lfloor \frac{\eta}{2} \rfloor$ )
8: RecursiveClustering( $T_2^*, O, \chi + \lfloor \frac{\omega}{2} \rfloor, \varphi, \omega - \lfloor \frac{\omega}{2} \rfloor, \lfloor \frac{\eta}{2} \rfloor$ )
9: RecursiveClustering( $T_3^*, O, \chi + \lfloor \frac{\omega}{2} \rfloor, \varphi + \lfloor \frac{\eta}{2} \rfloor, \omega - \lfloor \frac{\omega}{2} \rfloor, \eta - \lfloor \frac{\eta}{2} \rfloor$ )

```

Algorithm 3 Round-Robin Assignment

Input: Task set T^* , spatial coordinates of tasks O ,

Output: Set of subsets of tasks $\{T_0^*, T_1^*, T_2^*, T_3^*\}$

```

1:  $T_0^*, T_1^*, T_2^*, T_3^* \leftarrow \emptyset$ 
2:  $\{(X^-, Y^-, Z^-), (X^+, Y^+, Z^+)\} \leftarrow \text{BoundingBox}(O)$ 
3: Cluster cores  $\Psi \leftarrow \{(X^-, Y^-, Z^-), (X^-, Y^+, Z^-), (X^+, Y^-, Z^+), (X^+, Y^+, Z^+)\}$ 
4: while  $|T^*| > 0$  do
5:   for  $i = 0$  to 3 do
6:      $t \leftarrow \arg \min_{t \in T^*} \text{Distance}(O_t, \Psi_i)$ 
7:      $T_i^* \leftarrow T_i^* \cup t$ 
8:      $T^* \leftarrow T^* \setminus t$ 
9:   end for
10: end while
11: return  $\{T_0^*, T_1^*, T_2^*, T_3^*\}$ 

```

$\frac{1}{4}\omega\eta$. Then, MS-CLS determines the assignment from tasks to PEs by a round-robin heuristic as described in Algorithm 3. Specifically, each cluster grabs a task which is the closest to its core by turns, until all tasks have been assigned. In addition, if the width or height of the PE array is not the power of two, the capacity of the clusters will be different. Under such circumstances, we need to check if the number of tasks exceeds the capacity of a cluster and skip the assignment if there is overflow.

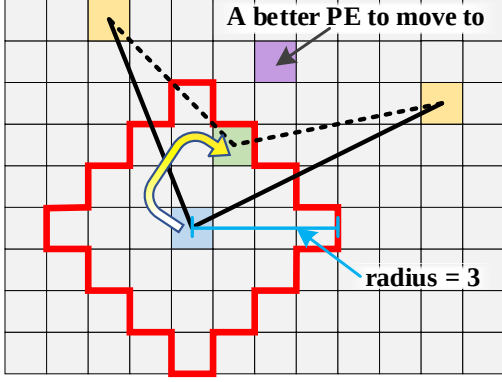


Fig. 8. The neighborhood structure. If we move the task placed in the blue PE to the green PE, we can reduce the wire length as it is closer to its neighbors placed in the orange PEs. The region enveloped by the red lines represents the destinations for all possible swap or move operations. A better PE (violet) to move to can be taken into account under a larger search radius.

E. Placement 2: Local Search for Placement Refinement

MS-CLS employs a local search-based metaheuristic algorithm to further optimize the initial placement according to the objective function as shown in Eq. (5). The proposed local search algorithm is based on swap and move neighborhood structures. We use $\text{Swap}(i, j)$ to denote swapping the assigned PEs of tasks i and j . If one of the PEs is empty, it reduces to a move operation $\text{Move}(i, j)$. We unify the descriptions by assuming that there is a dummy task j whose connections $C_j = \emptyset$. For each specific $\text{Swap}(i, j)$ or $\text{Move}(i, j)$, its improvement to the objective value $\Delta f_c(i, j)$ is calculated as Eq. (14).

$$\Delta f_c(i, j) = \sum_{k \in C_i} W_{e_j e_k} + \sum_{k \in C_j} W_{e_i e_k} - \sum_{k \in C_i} W_{e_i e_k} - \sum_{k \in C_j} W_{e_j e_k} \quad (14)$$

A typical local search implementation evaluates all possible neighborhood moves and performs the best one. However, in FEMPP, it is too time-consuming to evaluate all the swap and move operations regarding the huge number of PEs. Therefore, we only try to swap/move a task with/to the neighboring PEs. To be specific, the distance between the source and destination of a swap or move operation must be within a search radius B , as depicted in Fig. 8. Indeed, it is more promising to find a better operation under a larger search radius, but it will spend much more computational time. So, we need to reach a trade-off between effectiveness and efficiency. In MS-CLS, the search radius is set to $B = 2$.

Tabu search is a well-known metaheuristic algorithm introduced by Glover [36], which has been widely used to solve various combinatorial optimization problems. We adopt a tabu strategy which incorporates a recency-based tabu list to prohibit reverting recent operations to escape from the local optima and improve search efficiency. In detail, if a task t is moved at iteration i , it is forbidden to be moved again until iteration $i + L$, where L is the tabu tenure, a uniformly distributed random integer between 10 and 25. In terms of the

Algorithm 4 Local Search for Placement

Input: Task set T^* , initial placement e , search radius B

Output: Best solution found e^*

```

1:  $e^* \leftarrow e$ 
2: Iteration  $i \leftarrow 0$ 
3: Tabu list  $TL \leftarrow \emptyset$ 
4: repeat
5:   for each task  $t$  in task set  $T^*$  do
6:     if  $t \notin TL_i$  then
7:        $T_n^* \leftarrow \{t' \in T^* : W_{e_t e_{t'}} \leq B\}$ 
8:        $t' \leftarrow \arg \max_{t' \in T_n^*} \{\Delta f_c(t, t')\}$ 
9:        $e_t \leftrightarrow e_{t'}$  /* Perform Swap( $t, t'$ ) */
10:      if  $f_c(e) < f_c(e^*)$  then
11:         $e^* \leftarrow e$ 
12:      end if
13:      /*  $L \sim U(10, 25)$  */
14:       $TL_k \leftarrow TL_k \cup \{t\}, \forall k \in (i, i + L)$ 
15:    end if
16:  end for
17:   $i \leftarrow i + 1$ 
18: until time limit is reached

```

swap operation, it will be considered if any one of the tasks is not in tabu status, i.e., a task can be moved if another task needs to take its place, even if it is in tabu status. Algorithm 4 presents the procedure of the local search algorithm. At each iteration, all the tasks are investigated in a sequential manner, where for each task not in tabu status, the best swap or move operation which minimizes Δf_c in Eq. (14) is performed. That is, each task will be swapped or moved at most once at each iteration. The search procedure is repeated until the time limit is reached and the best solution found so far is returned.

V. EXPERIMENTAL RESULTS

A. Benchmarks and Setup

We implement MS-CLS in C++ programming language and compile it with Visual Studio 2017. In the ISPD 2021 contest, all the algorithms are executed on a 2.90 GHz AMD EPYC 7542 processor under a 50-minute time limit. In this paper, each instance is tested for ten runs in a single thread on an Intel Core i3-10100 CPU and 16GB RAM, under a conservative 30-minute time limit to reduce the bias caused by hardware³ and highlight the advantage of MS-CLS.

We conduct experiments based on the rules and datasets proposed in the ISPD 2021 contest. All benchmark instances and evaluation tools can be obtained from [37]. There are four datasets in total. Each instance models a 3D object in real-world applications, which are bullet, flange, propeller tip, and motorbike, respectively. The difficulty of solving an instance is affected by the shape of the object and the target resolution distribution. Fig. 9 illustrates the numbers of sample points which lie within different target resolution intervals in the heat map of each instance. From Fig. 9, it is obvious that

³According to <https://www.cpubenchmark.net>, the single thread performance of AMD EPYC 7542 and Intel Core i3-10100 are 2,206 and 2,645 MOps/Sec, respectively.

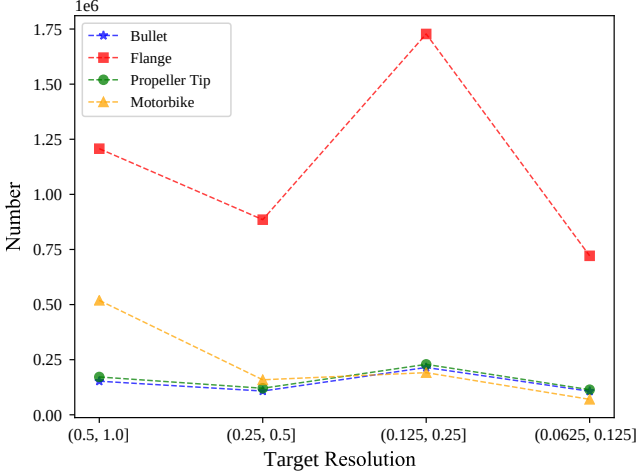


Fig. 9. The numbers of the sample points where the target resolution lies within the range of $(2^{-i}, 2^{1-i}]$ for each instance, where $i = 1, 2, 3, 4$.

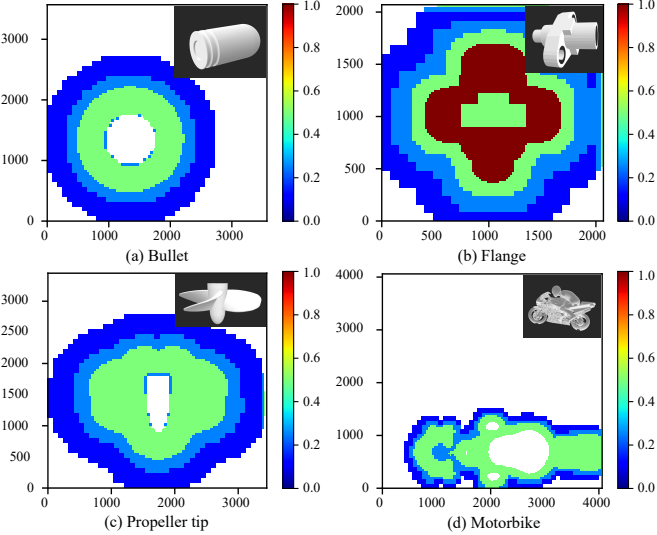


Fig. 10. Slices of the partitioning solutions. The top-right corner of each slice shows the corresponding 3D object, and the tiles with different actual resolutions in the partitioning results are colored with the corresponding colors in the right color bar.

the instance flange has a significantly higher overall target resolution than other ones, which makes it more challenging to optimize the accuracy using the same computing resources. Regarding placement optimization, the more overlaps there are when mapping the object from 3D to 2D, the more challenging it becomes. As depicted in Fig. 10(b), the shape of the flange instance is close to a cube, resulting in the most severe overlap. Due to these reasons, flange is the most challenging instance.

B. Summarized Results

Fig. 10 shows slices of the partitioning solutions obtained by MS-CLS, where warmer color represents the regions with higher actual resolutions. From the distribution of the tiles with different resolutions, we can see that the shape of the object is more or less reflected. Empirically, this fact indicates that

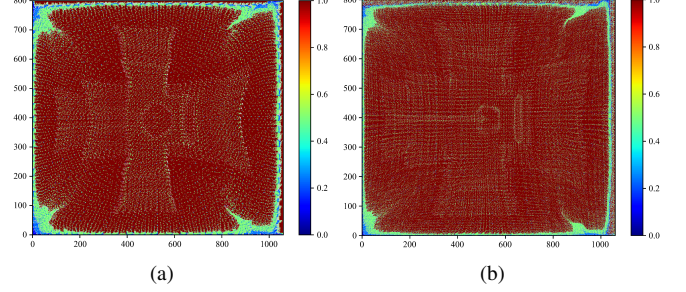


Fig. 11. Visualization of placement solution of instance flange. (a) The initial placement based on recursive clustering. The PEs corresponding to adjacent tiles in the partitioning result are clustered together in the neighboring region of the PE array. (b) The refined solution by swap-based tabu search. The adapter (colored in white) PEs are evenly dispersed after refinement to alleviate congestion and optimize wire length.

the actual resolution matches the target one, implying that the quality of the partitioning is relatively good.

Fig. 11 illustrates the placement solution of the instance flange. The color of each mesh PE is consistent with the color of the actual resolution in Fig. 10 and the adapter PEs are colored in white. The unused PEs are colored in black but we can barely see them since the utilization is high and unused PEs are very rare. As shown in Fig. 11(a), after mapping the flange object from 3D structure to 2D topology, the distribution of the PEs looks like the projection of the original shape to some extent. Meanwhile, adapters are gathered in the corresponding region where there are resolution differences. This phenomenon owes much to the consideration of maintaining the relative positions between the tasks during recursive clustering. The local search-based refinement further improves the placement of the PEs under the original QAP model. As illustrated in Fig. 11(b), it alleviates the local congestion by distributing the adapters in a more even manner, since an adapter has fewer connections than a tile.

C. Comparison with the Contestants

We compare the proposed MS-CLS⁴ with the top six contestants in the ISPD 2021 contest (denoted by ISPD#1/2/...) [38] and the state-of-the-art algorithm [16] (denoted by GEBGP) in the literature, which are introduced in Section I. Table I presents the statistics of the computational results obtained by different algorithms. Columns Bullet, Flange, Propeller Tip, and Motorbike report the evaluation scores of the corresponding benchmark instances, respectively. Column f_a gives the accuracy score and column f_c represents the connectivity score. The final objective value of an instance is $f = \min\{f_a, f_c\}$. Column Total Score gives the sum of the objective values over all test cases. The numbers in bold indicate the best score among the listed algorithms on the corresponding instances.

Table I shows that MS-CLS achieves a 2.56% improvement in the final score comparing to the best-known result and outperforms all the reference algorithms in terms of the subscores on three out of four instances. For the connectivity score,

⁴The source code of MS-CLS is available at <https://github.com/lxily/MS-CLS>.

TABLE I
COMPARISON WITH OTHER CONTESTANTS IN ISPD 2021 CONTEST REGARDING ACCURACY AND CONNECTIVITY

Algorithm	Timeout (minute)	Total Score	Bullet			Flange			Propeller Tip			Motorbike		
			f_a	f_c	f	f_a	f_c	f	f_a	f_c	f	f_a	f_c	f
ISPD#1	50	2.4370	0.6140	0.7917	0.6140	0.4614	0.5810	0.4614	0.7089	0.7245	0.7089	0.6528	0.7706	0.6528
ISPD#2	50	2.3642	0.5194	0.6771	0.5194	0.4689	0.6994	0.4689	0.7089	0.8102	0.7089	0.6670	0.8764	0.6670
ISPD#3	50	2.0033	0.4756	0.5063	0.4756	0.3883	0.7599	0.3883	0.7088	0.4905	0.4905	0.6516	0.6489	0.6489
ISPD#4	50	1.8033	0.2608	0.7777	0.2608	0.4743	0.5814	0.4743	0.6539	0.6536	0.6536	0.5555	0.4146	0.4146
ISPD#5	50	1.5919	0.3648	0.5685	0.3648	0.3849	0.5341	0.3849	0.5585	0.5618	0.5585	0.2837	0.8539	0.2837
ISPD#6	50	0.5241	0.1665	0.1636	0.1636	0.1889	0.1789	0.1789	0.1049	0.4163	0.1049	0.3116	0.0767	0.0767
GEBGP [†]	50	2.5367	0.6890	0.7952	0.6890	0.4741	0.6898	0.4741	0.7089	0.7409	0.7089	0.6647	0.9577	0.0767
MS-CLS	30	2.5432	0.6768	0.8132	0.6768	0.4809	0.7457	0.4809	0.7152	0.8494	0.7152	0.6703	1.0116	0.6703
MS-CLS	10	2.5432	0.6768	0.7810	0.6768	0.4809	0.7186	0.4809	0.7152	0.8330	0.7152	0.6703	0.9905	0.6703

[†] Tested on a 2.8GHz Intel Xeon E5-2680 CPU.

our average accuracy achieves 4.51% improvement compared with the first place solver, and achieves 1.06% improvement on average on all instances except for bullet compared with the best reference algorithm GEBGP. As shown in Fig. 10, the instance bullet is relatively special comparing to other instances, e.g., its surface is composed of simple shapes and it is both central and mirror symmetric. The reason for the disadvantage on the instance bullet may be that MS-CLS is tuned for general cases and does not exploit such special properties. As for connectivity, MS-CLS outperforms all contestants on all benchmarks. It demonstrates a clear advantage on instances flange and propeller tip, where the improvement reaches 8.10% and 14.64% comparing to GEBGP, respectively.

Regarding instance flange, the preliminary version of our proposed MS-CLS, which won the third place in the contest, obtained the highest connectivity score. The reason is that its poor partitioning result increases the upper bound and decreases the complexity of the placement optimization, as discussed in Section III-D. However, the accuracy score is usually the bottleneck of the final solution quality under the current metric. Compared with our preliminary version in solving the partitioning problem in the contest, MS-CLS enhances the merging strategy to avoid a mess of tile distribution and improves the evaluation metrics of the split operation to reduce the use of adapters. These strategies enable MS-CLS to support larger inverse sampling step for finer partitioning, which improves accuracy significantly.

Apart from the solution quality, even when the time is limited to ten minutes, MS-CLS can still outperform all the competitors in terms of all the subscores on three out of four instances, as shown in Table I. In summary, the experimental results demonstrate both the effectiveness and efficiency of the proposed MS-CLS algorithm.

VI. ANALYSIS AND DISCUSSION

A. Importance of Inverse Sampling Step Estimation

The inverse sampling step s needs to be determined carefully as it profoundly affects the quality of the partitioning solution. Different from GEBGP which traverses all possible sampling steps under physical memory limitation, our proposed MS-CLS estimates a reasonable upper bound of s

TABLE II
ISPD 2021 CONTEST BENCHMARK CHARACTERISTICS

Instance	Dimension	Candidate inverse sampling step s
Bullet	$119 \times 119 \times 119$	10, 20, 30, ...
Flange	$207 \times 207 \times 207$	10, 20, 30, ...
Propeller Tip	$115 \times 115 \times 115$	2, 4, 6, 8, 10, 12, ...
Motorbike	$203 \times 203 \times 203$	10, 20, 30, ...

according to the actual resolution of the heatmap and evaluates possible values in decreasing order.

In order to satisfy the seamless tiling constraints, the number of cells in each dimension of the target space needs to be an integral multiple of the processing capacity M of the PE ($M = 10$ in the current WSE, see Section III). According to the dimension of the target space reported in Table II, the candidate inverse sampling step of instances bullet, flange, and motorbike must be a multiple of 10. However, there are more choices for candidate s of instance propeller tip, i.e., all even numbers are feasible since the number of unit steps in each dimension is 115 which already has a factor of 5. Consequently, the number of feasible s is much larger than other three instances, which makes it very time-consuming to iterate over all possible values of s . Apparently, it is unnecessary to try s values that are too small, because it will be hard to make full use of the available PEs, which usually results in poor solution quality. Specifically, on instance propeller tip, GEBGP chooses $s = 16$, while our proposed MS-CLS chooses $s = 30$ using the method proposed in Section IV-B, which results in a better partitioning. The above discussion justifies the effectiveness of the proposed inverse sampling step estimation technique.

B. Effectiveness of Different Splitting Operation Metrics

Both MS-CLS and GEBGP employ a merging-splitting-based strategy to optimize accuracy. The major difference between them is the evaluation of splitting operations. While GEBGP prefers the tiles with the maximal ratio of target resolution to actual resolution to split, MS-CLS considers the effect of both the tiles and the adapters when selecting splitting operations. For convenience, we refer to the proposed

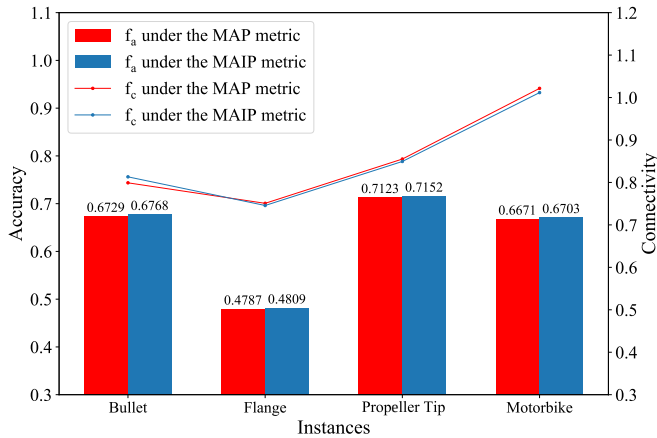


Fig. 12. Accuracy and connectivity scores of enabling MAIP and MAP.

strategy as maximum average improvement priority (MAIP), and denote the one used by GEBGP as maximum accuracy priority (MAP). We compare their influence on partitioning and placement by replacing MAIP with MAP in MS-CLS. Specifically, we modify line 11 of Algorithm 1 to Eq. (15).

$$t^* \leftarrow \arg \max_{t \in T} \{f_a(t)\} \quad (15)$$

Fig. 12 compares the accuracy and connectivity scores obtained by enabling MAIP and MAP. Fig. 13 shows the resolution distribution of PEs in the resulting partition solution for each instance. From Fig. 13, we can observe that both MAIP and MAP can make use of all available PEs to improve accuracy. When replacing MAIP with MAP, the number of adapters in the partitioning solution of all instances increases by 0.3% ~ 1.0%, and the accuracy score decreases accordingly, as analyzed in Section III-D. However, the increment of adapters reduces the difficulty of placement optimization. Therefore, the connectivity scores are slightly improved except for the instance bullet as illustrated in Fig. 12. On the instance bullet, the increase in the number of adapters under the MAP strategy is less than that of other instances (0.3%, the other three ones are 0.4%, 0.6%, and 1.0%, respectively) as illustrated in Fig. 13. However, the number of mesh PEs with different resolutions is significantly different from that of the MAIP strategy. The impact of structural changes outweighs the benefits of the increased number of adapters, eventually leading to a slight decrease in connectivity on the instance bullet.

C. Comparison of Initial Placement and Refinement

We compare the placement optimization of our proposed MS-CLS algorithm with GEBGP, which mainly follows the optimization framework of the placement in the VLSI design flow. Specifically, it regards the connection graph between tasks as a netlist, where tasks and connections correspond to standard cells and nets. Despite the classic global placement algorithm ePlace [17], GEBGP performs geometric bisection based on the spatial location of tiles so that all of them are mapped to the PE array in the form of a snaking chain to generate the initial layout. Then, the better placement between

TABLE III
COMPARISON OF INITIAL PLACEMENT AND REFINEMENT WITH GEBGP

Instance	GEBGP			MS-CLS		
	f_c^b	f_c^e	f_c^r	f_c^i	f_c^m	f_c^r
Bullet	0.6562	0.7912	0.7952	0.6309	0.7810	0.8132
Flange	0.5954	0.6880	0.6898	0.6081	0.7186	0.7457
Propeller Tip	0.7409	0.6839	0.7140	0.6816	0.8330	0.8494
Motorbike	0.7234	0.9477	0.9577	0.7734	0.9905	1.0116
Avg.	0.6790	0.7777	0.7892	0.6735	0.8308	0.8550

ePlace and the geometric bisection is passed to the legalization and refinement to produce the final placement.

In Table III, f_c^b represents the connectivity score of the geometric bisection in GEBGP. f_c^e reports the connectivity score obtained by ePlace. f_c^i and f_c^r give the initial and refined connectivity scores, respectively. In addition, we present the intermediate results of MS-CLS running for 5 minutes, denoted by f_c^m . From Table III, we can observe that the average performance of our clustering-based initial placement matches the geometric bisection of GEBGP. The geometric bisection obtains better results on the instances bullet and propeller tip which are symmetric, but MS-CLS performs better on the irregular instances. The ePlace-based placement obtains the best initial connectivity scores, but its improvement from f_c^i to f_c^r is only 1.56%. On the contrary, MS-CLS achieves 21.04% improvement starting from f_c^i and finally gets the highest connectivity score on all the instances. Furthermore, when the time limit is 5 minutes, which is much shorter than the one used in the ISPD 2021 contest, MS-CLS can still outperform GEBGP on all the instances except for the instance bullet, which justifies the effectiveness of the proposed algorithm.

D. Analysis on Recursive Clustering

As discussed in Section III-D, it should be beneficial to preserve the relative positions between tiles and adapters when mapping them from the 3D target space to the 2D PE array, and our recursive clustering placement method is based on this assumption. In order to verify the effectiveness and efficiency of the proposed algorithm, we further compare the evolution of the objective value over time among four initial placement methods on instances Propeller Tip and Motorbike as depicted in Fig. 14. In this figure, R4C stands for recursive 4-clustering, which is the method used in the proposed MS-CLS algorithm to obtain the results in Table I. R2C represents recursive 2-clustering, where the only difference between R2C and R4C is that, R2C bisects the tasks instead of quartering them, which is similar to GEBGP. Method Greedy places the tasks one by one to the PE array, and for each task, it is placed to a PE which minimizes the total wire length to its placed neighboring tasks. Finally, method Random just iteratively places each task on a random empty PE. We incorporate these four initial placement methods in our MS-CLS algorithm while keeping other components unchanged.

From Fig. 14, it is easy to observe that random placement is completely impractical due to its poor results. Since it does not take the wire length into account, the initial placement is

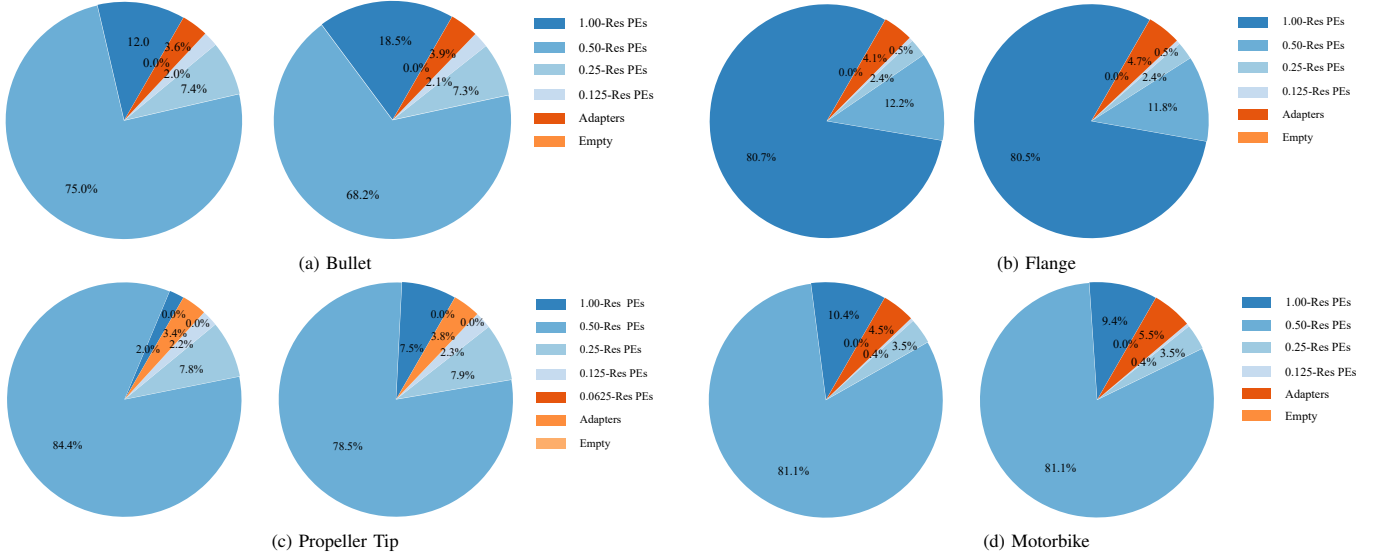


Fig. 13. Resolution distribution of PEs in partitioning solution. For each instance, the left (right) pie chart represents the result of enabling MAIP (MAP).

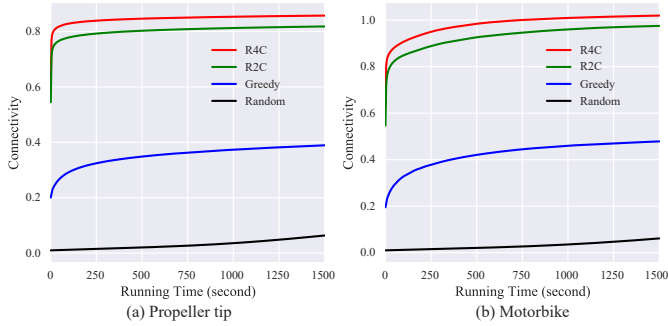


Fig. 14. Evolution of the connectivity score over time for different initial placement methods on two representative instances.

in a mess such that the following local search is powerless to improve the solution quality. As for greedy placement, although it considers the connections among tasks from a local point of view, the global layout is barely considered. Thus, the local search easily converges to medium-quality local optimal solutions. However, remarkable improvement can be obtained when the recursive clustering is employed. It is noteworthy that the connectivity scores of the initial solutions obtained by R4C and R2C are even better than the ones optimized by local search based on the greedy and random placements.

Apart from the initial solution quality, the recursive clustering can also enhance the local search procedure. As shown in Fig. 14, the solution quality improves dramatically in the first local search iterations. In fact, it only takes a few minutes for MS-CLS to surpass the best results of all other competitors. Furthermore, the phenomenon that R4C dominates R2C by a slight advantage reveals that the solution quality is not very sensitive to the parameter of the cluster number. However, quartering is more promising than bisecting. The reason might be that finer-grained clustering is closer to the original placement problem. In summary, these statistics confirm that the recursive clustering is important for the effectiveness and efficiency of MS-CLS.

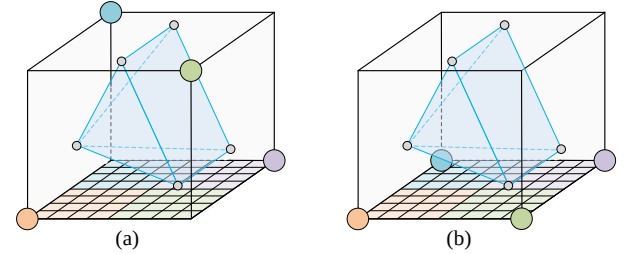


Fig. 15. Two different patterns of cluster core location. (a) Disperse the cluster cores as much as possible. (b) Put the cluster cores on the same surface of the bounding box.

TABLE IV
COMPARISON OF DIFFERENT ATTRACTION SOURCE LOCATIONS

Instance	MS-CLS ^f		MS-CLS ^c		MS-CLS	
	f_c^i	f_c	f_c^i	f_c	f_c^i	f_c
Bullet	0.5625	0.8104	0.6085	0.8121	0.6309	0.8132
Flange	0.5461	0.7451	0.5876	0.7461	0.6081	0.7457
Propeller Tip	0.6188	0.8481	0.6618	0.8487	0.6816	0.8494
Motorbike	0.6988	1.0102	0.7521	1.0103	0.7734	1.0116
Avg.	0.6066	0.8535	0.6525	0.8543	0.6735	0.8550
Norm.	90.06%	99.82%	96.88%	99.92%	100.00%	100.00%

E. Discussion on Cluster Core Location

We further compare different cluster core location strategies in our proposed cluster-based initial placement algorithm. In Fig. 15(a), the cluster cores are dispersed as much as possible such that the preference from tasks to clusters will be diversified. However, the location of the cluster cores may not fit into the structure of the PE array since they are not on the same plane. An alternative pattern which fits into the structure of the PE array is shown in Fig. 15(b), but the cluster cores are closer to each other than the pattern used in MS-CLS. In Table IV, MS-CLS^f and MS-CLS^c adopt the clustering patterns corresponding to Figs. 15(a) and 15(b), while other components remain unchanged. We can

observe that the cluster core location results in a noticeable difference to the initial placement and the pattern adopted by MS-CLS obtains much better results. High-quality initial placement usually makes the subsequent refinement quickly converge to high-quality local optima. However, regardless of the computational efficiency, the gaps between the final connectivity scores of these configurations are relatively small, where MS-CLS has only 0.18% and 0.08% improvement over MS-CLS^f and MS-CLS^c, respectively. These statistics indicate that the proposed refinement method is effective and not sensitive to the cluster core location.

VII. CONCLUSION

In this work, we propose the MS-CLS algorithm for the partitioning and placement problem in hardware-accelerated physics modeling, which is the topic of the ISPD 2021 contest. The proposed MS-CLS algorithm improves the accuracy of the FEM computation on WSE by bottom-up non-uniform iterative partitioning. For the massive simulation tasks derived from FEM modeling, MS-CLS explores the implicit correlation between the computational graph and the Network-on-Chip fabric of WSE. Then, it integrates a spatial recursive cluster-based heuristic to optimize the placement of tasks and improves the connectivity between parallel processors. According to the experiments, MS-CLS is highly competitive in terms of both solution quality and computational time compared to the best results of all the qualified teams in the ISPD 2021 contest and the state-of-the-art solver GEBGP in the literature. In the future, we will try to generalize the ideas of the recursive clustering techniques to tackle other task placement and parallel computing problems.

ACKNOWLEDGMENTS

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62202192, 72101094, and the Special Project for Knowledge Innovation of Hubei Province under Grant 2022013301015175.

REFERENCES

- [1] E. Süli, "Lecture notes on finite element methods for partial differential equations," *Mathematical Institute, University of Oxford*, 2012.
- [2] R. W. Clough, "The finite element method in plane stress analysis," in *Proceedings of 2nd ASCE Conference on Electronic Computation*, 1960, pp. 345–377.
- [3] M. Zolgharni, P. D. Ledger, D. W. Armitage *et al.*, "Imaging cerebral haemorrhage with magnetic induction tomography: Numerical modelling," *Physiological Measurement*, vol. 30, no. 6, p. S187, 2009.
- [4] G. R. Liu, K. Y. Dai, and T. T. Nguyen, "A smoothed finite element method for mechanics problems," *Computational Mechanics*, vol. 39, no. 6, pp. 859–877, 2007.
- [5] F. Klocke, T. Beck, S. Hoppe *et al.*, "Examples of FEM application in manufacturing technology," *Journal of Materials Processing Technology*, vol. 120, no. 1–3, pp. 450–457, 2002.
- [6] V. Jagota, A. P. S. Sethi, and K. Kumar, "Finite element method: An overview," *Walailak Journal of Science and Technology (WJST)*, vol. 10, no. 1, pp. 1–8, 2013.
- [7] Z. A. Taylor, M. Cheng, and S. Ourselin, "High-speed nonlinear finite element analysis for surgical simulation using graphics processing units," *IEEE Transactions on Medical Imaging*, vol. 27, no. 5, pp. 650–663, 2008.
- [8] A. Dziekonski, A. Lamecki, and M. Mrozowski, "GPU acceleration of multilevel solvers for analysis of microwave components with finite element method," *IEEE Microwave and Wireless Components Letters*, vol. 21, no. 1, pp. 1–3, 2010.
- [9] H. Sellami, L. Cazenille, T. Fujii *et al.*, "Accelerating the finite-element method for reaction-diffusion simulations on GPUs with CUDA," *Micromachines*, vol. 11, no. 9, p. 881, 2020.
- [10] P. Groeneveld, M. James, V. Kibardin *et al.*, "ISPD 2021 wafer-scale physics modeling contest: A new frontier for partitioning, placement and routing," in *ISPD*, 2021, pp. 143–147.
- [11] T. Wei, J. Zhong, and J. Zhang, "An energy-efficient partition-based framework with continuous ant colony optimization for target tracking in mobile sensor networks," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 5, no. 4, pp. 700–713, 2019.
- [12] X. Xu, S. Fu, W. Li, F. Dai, H. Gao, and V. Chang, "Multi-objective data placement for workflow management in cloud infrastructure using NSGA-II," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 4, no. 5, pp. 605–615, 2020.
- [13] Z. Su, Q. Zhang, Z. Lü *et al.*, "Weighting-based variable neighborhood search for optimal camera placement," in *AAAI*, May 2021, pp. 12 400–12 408.
- [14] C. Zhang and J. A. Shah, "Co-optimizing multi-agent placement with task assignment and scheduling," in *IJCAI*, 2016, p. 3308–3314.
- [15] ISPD, "ISPD 2021: Wafer-scale physics modeling contest announcements," <https://www.ispd.cc/contests/21/index.html>, 2021.
- [16] Y. Lin, R. Liang, Y. Li, H. Hu, and J. Hu, "Mapping large scale finite element computing on to wafer-scale engines," in *ASP-DAC*. IEEE, 2022, pp. 147–153.
- [17] J. Lu, H. Zhuang, P. Chen, H. Chang, C.-C. Chang, Y.-C. Wong, L. Sha, D. Huang, Y. Luo, C.-C. Teng *et al.*, "ePlace-MS: Electrostatics-based placement for mixed-size circuits," *TCAD*, vol. 34, no. 5, pp. 685–698, 2015.
- [18] T. C. Chen, Z. W. Jiang, T. C. Hsu, H. C. Chen, and Y. W. Chang, "NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 7, pp. 1228–1240, 2008.
- [19] A. Mirhoseini, A. Goldie, M. Yazgan *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.
- [20] M. D. Moffitt, M. A. Sustik, and P. G. Villarrubia, "Robust partitioning for hardware-accelerated functional verification," in *DAC*, 2011, pp. 854–859.
- [21] M. D. Moffitt, G. E. Günther, and K. A. Pasnik, "Place and route for massively parallel hardware-accelerated functional verification," in *ICCAD*, 2013, pp. 466–472.
- [22] K. Rocki, D. Van E., I. Sharapov *et al.*, "Fast stencil-code computation on a wafer-scale processor," in *SC*, 2020.
- [23] M. V. Trotter, C. Q. Nguyen, S. Young, R. T. Woodruff, and K. M. Branson, "Epigenomic language models powered by cerebrar," *arXiv preprint arXiv:2112.07571*, 2021.
- [24] B. Jiang, J. Chen, J. Liu *et al.*, "CU.POKer: Placing DNNs on wafer-scale ai accelerator with optimal kernel sizing," in *ICCAD*, 2020, pp. 1–9.
- [25] Z. He, P. Liao, S. Liu *et al.*, "Physical synthesis for advanced neural network processors," in *ASP-DAC*, 2021, pp. 833–840.
- [26] J. Ding, L. Shen, Z. Lü, L. Xu, and U. Benlic, "A hybrid memetic algorithm for the parallel machine scheduling problem with job deteriorating effects," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 4, no. 3, pp. 385–397, 2019.
- [27] T.-W. Huang, D.-L. Lin, C.-X. Lin, and Y. Lin, "Taskflow: A lightweight parallel and heterogeneous task graph computing system," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 6, pp. 1303–1320, 2021.
- [28] J. Mack, S. E. Arda, U. Y. Ogras, and A. Akoglu, "Performant, multi-objective scheduling of highly interleaved task graphs on heterogeneous system on chip devices," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 9, pp. 2148–2162, 2021.
- [29] Y. Wang, W. Yang, A. P. Punnen, J. Tian, A. Yin, and Z. Lü, "The rank-one quadratic assignment problem," *INFORMS Journal on Computing*, vol. 33, no. 3, pp. 979–996, 2021.
- [30] A. B. Kahng, S. Reda, and Q. Wang, "Architecture and details of a high quality, large-scale analytical placer," in *ICCAD*. IEEE, 2005, pp. 891–898.
- [31] A. E. Caldwell, A. B. Kahng, and I. L. Markov, "Can recursive bisection alone produce routable placements?" in *DAC*, 2000, pp. 477–482.

- [32] S.-W. Hur and J. Lillis, "Relaxation and clustering in a local search framework: application to linear placement," *VLSI Design*, vol. 14, no. 2, pp. 143–154, 2002.
- [33] M. Pan, N. Viswanathan, and C. Chu, "An efficient and effective detailed placement algorithm," in *ICCAD*. IEEE, 2005, pp. 48–55.
- [34] C. Chen and K.-C. Tsai, "The server reassignment problem for load balancing in structured p2p systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 19, no. 2, pp. 234–246, 2008.
- [35] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.
- [36] F. Glover, "Future paths for integer programming and links to artificial intelligence," *Computers & operations research*, vol. 13, no. 5, pp. 533–549, 1986.
- [37] Cerebras, "Benchmarks and evaluation tools for the ISPD 2021 wafer-scale physics modeling contest," <https://github.com/Cerebras/ispd2021>, 2021.
- [38] G. Possler, "Session details: Session 13: Contest and results," in *ISPD*, 2021.



Canhui Luo received his B.E. degree in Internet of things Engineering from Hunan University, Changsha, China, in 2020. He is currently pursuing the Ph.D. degree in computer science and technology from the College of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China. His research focuses on metaheuristics for solving NP-hard combinatorial optimization problems, especially placement and routing in physical design and scheduling optimization on real-world applications.



Zhouxing Su received his B.E. degree in computer science and technology from Huazhong University of Science and Technology (HUST), China, in 2014, and his Ph.D. degree in computer software and theory from Laboratory of Smart Computing and Optimization (SMART) at HUST, in 2020. His research focuses on using metaheuristics to tackle classical NP-hard problems and real-world applications, such as placement, assignment, routing, scheduling, packing, covering, location, timetabling, and rostering.



Zhipeng Lü received the BS degree in applied mathematics from Jilin University, China, in 2001, and the PhD degree in computer software and theory from the Huazhong University of Science and Technology, China, in 2007. He was a Postdoctoral Research Fellow with the LERIA, Department of Computer Science, University of Angers, France, from 2007 to 2011. He is currently a Professor with the School of Computer Science and Technology, Huazhong University of Science and Technology, and the Director of the Institute of Artificial Intelligence and Optimization. His research interests include electronic design automation, computational intelligence, applied optimization, scheduling and planning, operations research and adaptive metaheuristics for solving large-scale real-world and theoretical combinatorial optimization, and constrained satisfaction problems.